CHAPTER 2
First ASP.NET Core Application

CONTENT

- Setting the Scene and Create Project
- Adding a Data Model

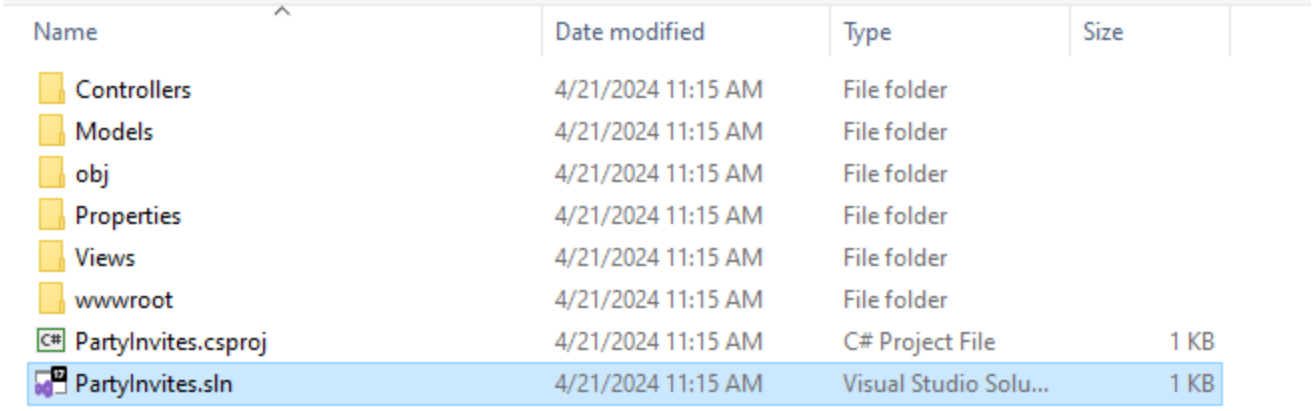
SETTING THE SCENE

Setting the Scene

- Imagine that a friend has decided to host a New Year's Eve party and that she has asked me to create a web app that allows her invitees to electronically RSVP. She has asked for these four key features:
  - A home page that shows information about the party
  - A form that can be used to RSVP
  - Validation for the RSVP form, which will display a thank-you page
  - A summary page that shows who is coming to the party

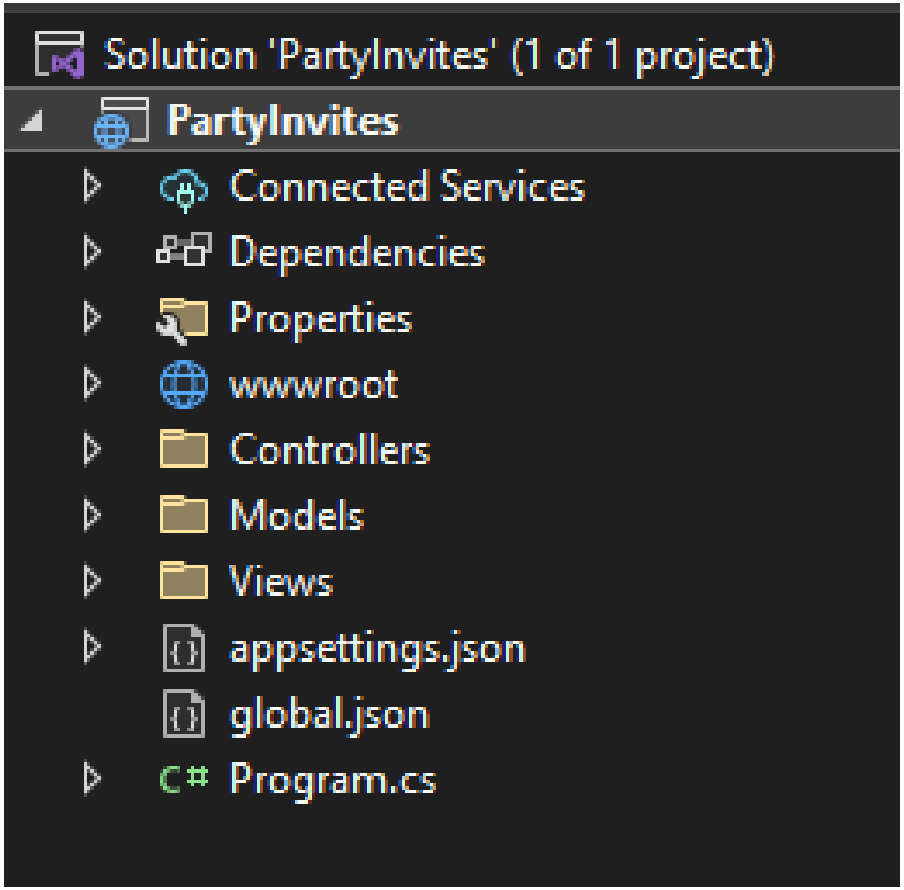
Create Project

- Use cmd again and go to the folder we want to create new project
- Run these commands below:
  - dotnet new globaljson --sdk-version 6.0.100 --output PartyInvites
  - dotnet new mvc --no-https --output PartyInvites --framework net6.0
  - dotnet new sln -o PartyInvites
  - dotnet sln PartyInvites add PartyInvites



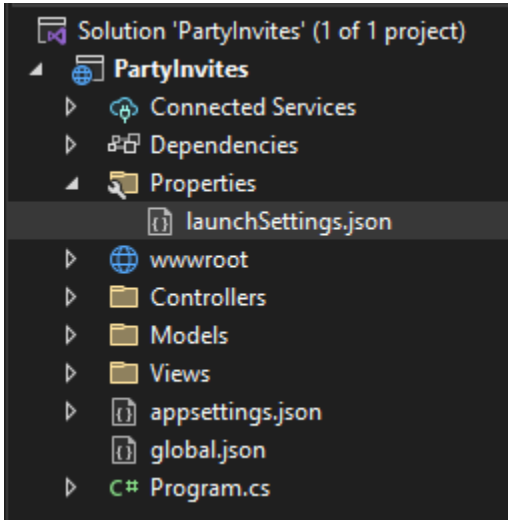
Name	Date modified	Type	Size
Controllers	4/21/2024 11:15 AM	File folder	
Models	4/21/2024 11:15 AM	File folder	
obj	4/21/2024 11:15 AM	File folder	
Properties	4/21/2024 11:15 AM	File folder	
Views	4/21/2024 11:15 AM	File folder	
wwwroot	4/21/2024 11:15 AM	File folder	
PartyInvites.csproj	4/21/2024 11:15 AM	C# Project File	1 KB
PartyInvites.sln	4/21/2024 11:15 AM	Visual Studio Solu...	1 KB

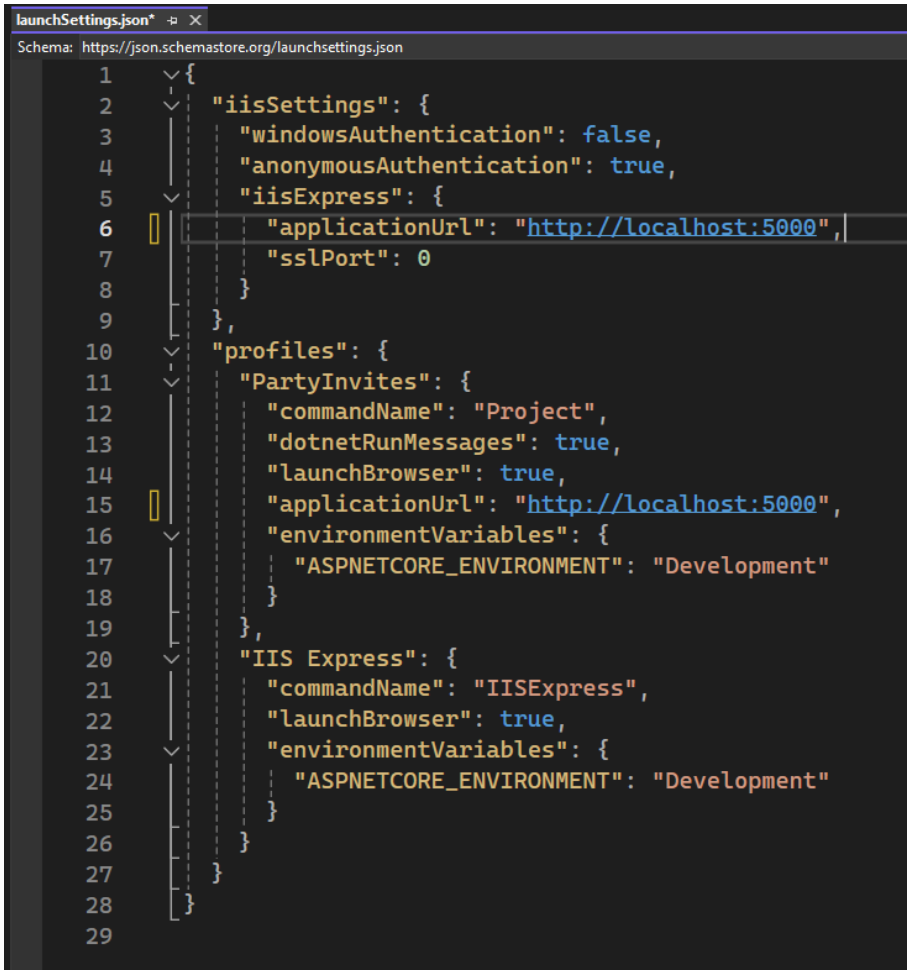
Create Project



Prepare Project

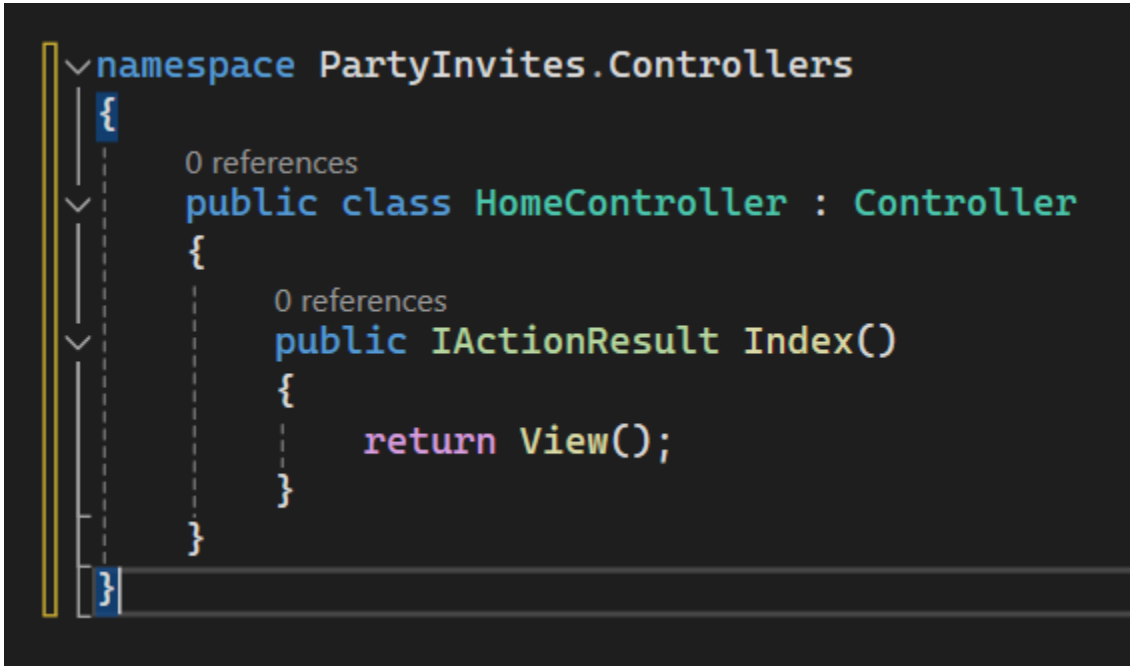
- Open lauchSettings.json and change ports to 5000





Prepare Project

- Replace the contents of the HomeController.cs file in the Controllers folder with the code below



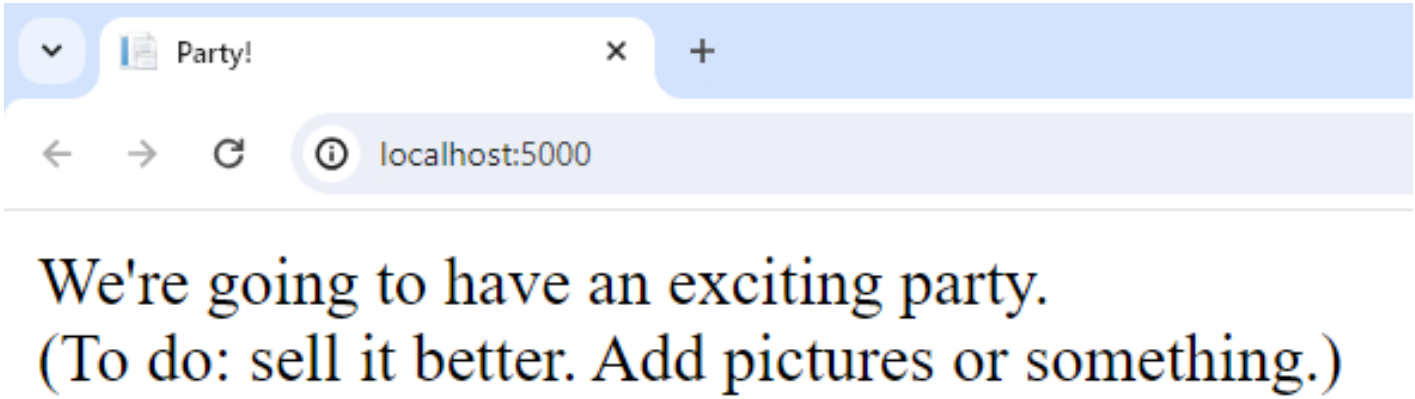
Prepare Project

- Replacing the Contents of the Index.cshtml File in the Views/Home Folder as below:

```
@{
 Layout = null;
 }
<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title>Party!</title>
</head>
<body>
    <div>
        <div>
            We're going to have an exciting party.<br />
            (To do: sell it better. Add pictures or something.)
        </div>
    </div>
</body>
</html>
```

Prepare Project

- Press F5 and see the result



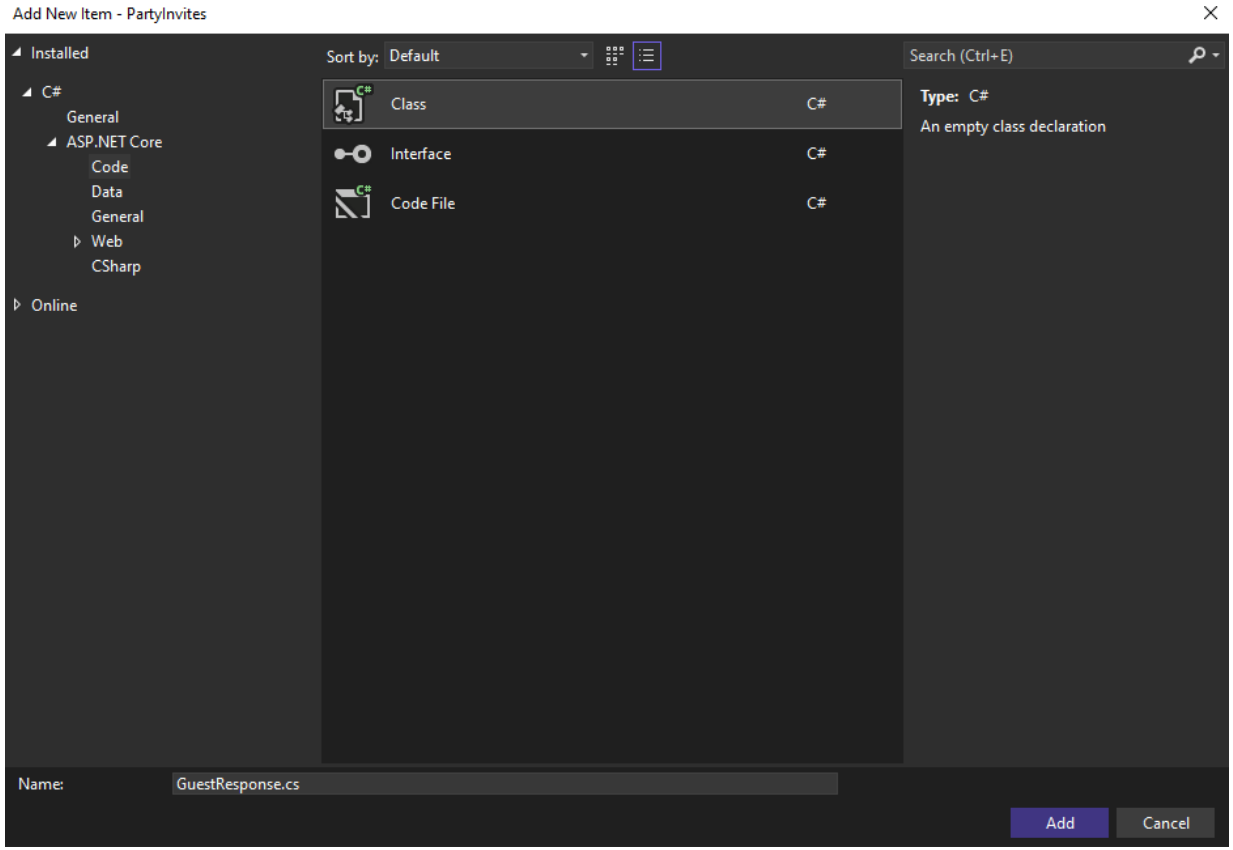
ADDING A DATA MODEL

Adding a Data Model

- The data model is the most important part of any ASP.NET Core application. The model is the **representation of the real-world objects, processes, and rules that define the subject**, known as the domain, of the application.
- The model, often referred to as a **domain model**, contains the C# objects (known as domain objects) that make up the universe of the application and the methods that manipulate them.
- In most projects, the job of the ASP.NET Core application is to provide the user with access to the data model and the features that allow the user to interact with it.
- The convention for an ASP.NET Core application is that the data model classes are defined **in a folder named Models**

Adding a Data Model

- If you are using Visual Studio, right-click the Models folder and select Add ➤ Class from the pop-up menu. Set the name of the class to GuestResponse.cs and click the Add button



Adding a Data Model

- If you are using Visual Studio, right-click the Models folder and select Add ➤ Class from the pop-up menu. Set the name of the class to GuestResponse.cs and click the Add button

```
namespace PartyInvites.Models
{
    public class GuestResponse
    {
        public string? Name { get; set; }
        public string? Email { get; set; }
        public string? Phone { get; set; }
        public bool? WillAttend { get; set; }
    }
}
```

CREATING A SECOND ACTION AND VIEW

Creating a Second Action and View

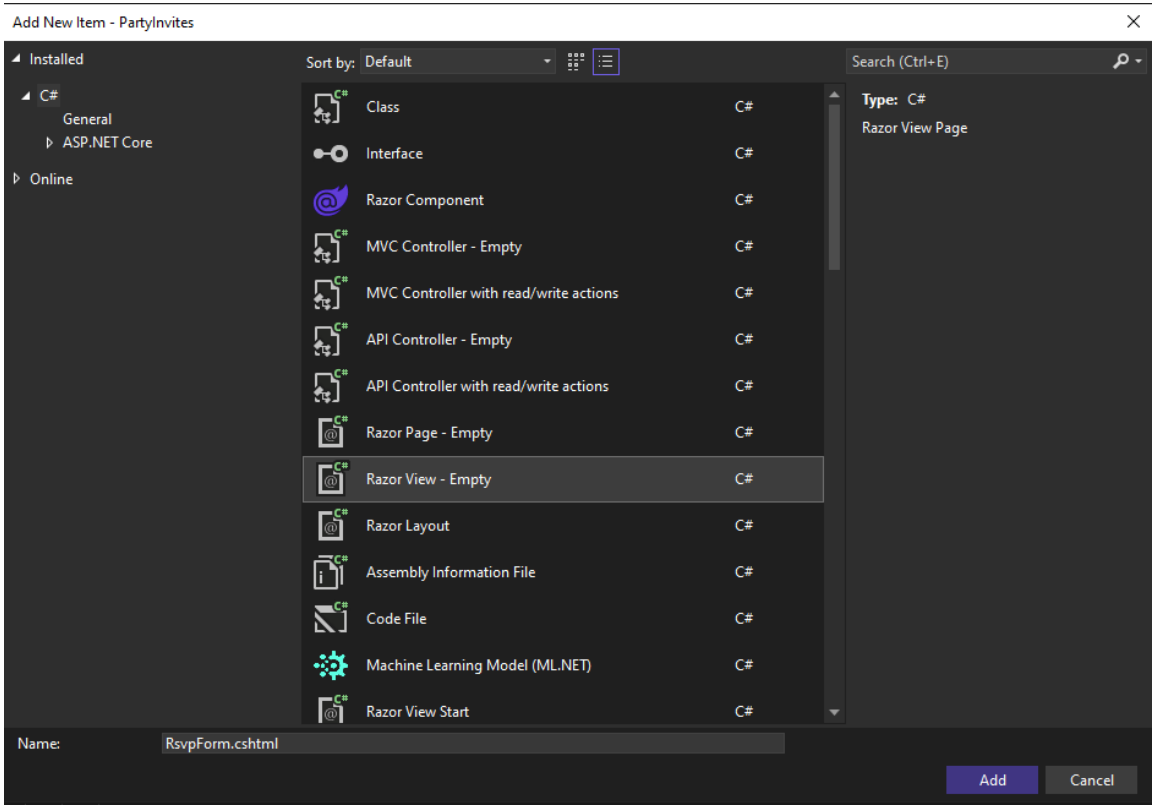
- Adding an Action Method in the HomeController.cs File in the Controllers Folder

```
namespace PartyInvites.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }

        public ViewResult RsvpForm()
        {
            return View();
        }
    }
}
```

Creating a Second Action and View

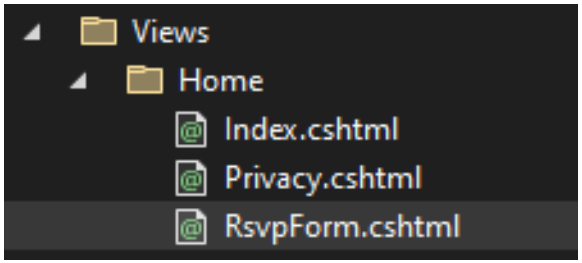
- If you are using Visual Studio, right-click the Views/Home folder and select Add ➤ New Item from the pop-up menu. Select the Razor View - Empty item, set the name to RsvpForm.cshtml, and click the Add button to create the file.



Creating a Second Action and View

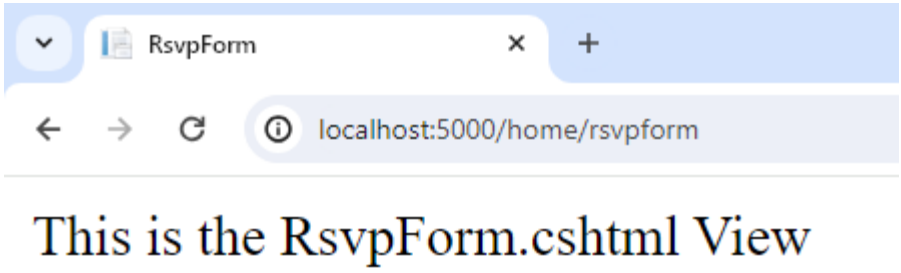
- Content of the new created View RsvpForm.cshtml

```
@{
    Layout = null;
}
<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title>RsvpForm</title>
</head>
<body>
    <div>
        This is the RsvpForm.cshtml View
    </div>
</body>
</html>
```



Creating a Second Action and View

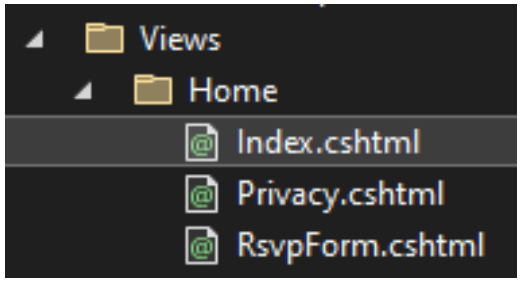
- Use the browser to request http://localhost:5000/home/rsvpform



LINKING ACTION METHODS

Linking Action Methods

- Create a link from the Index view so that guests can see the RsvpForm view without having to know the URL that targets a specific action method

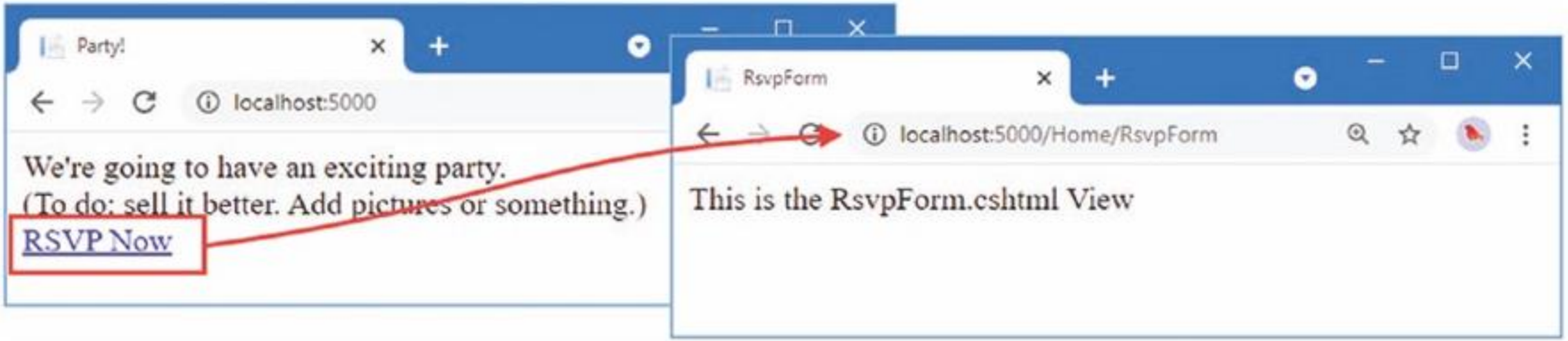


Linking Action Methods

```
@{
 Layout = null;
 }
<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title>Party!</title>
</head>
<body>
    <div>
        <div>
            We're going to have an exciting party.<br />
            (To do: sell it better. Add pictures or something.)
        </div>
        <a asp-action="RsvpForm">RSVP Now</a>
    </div>
</body>
</html>
```

Linking Action Methods

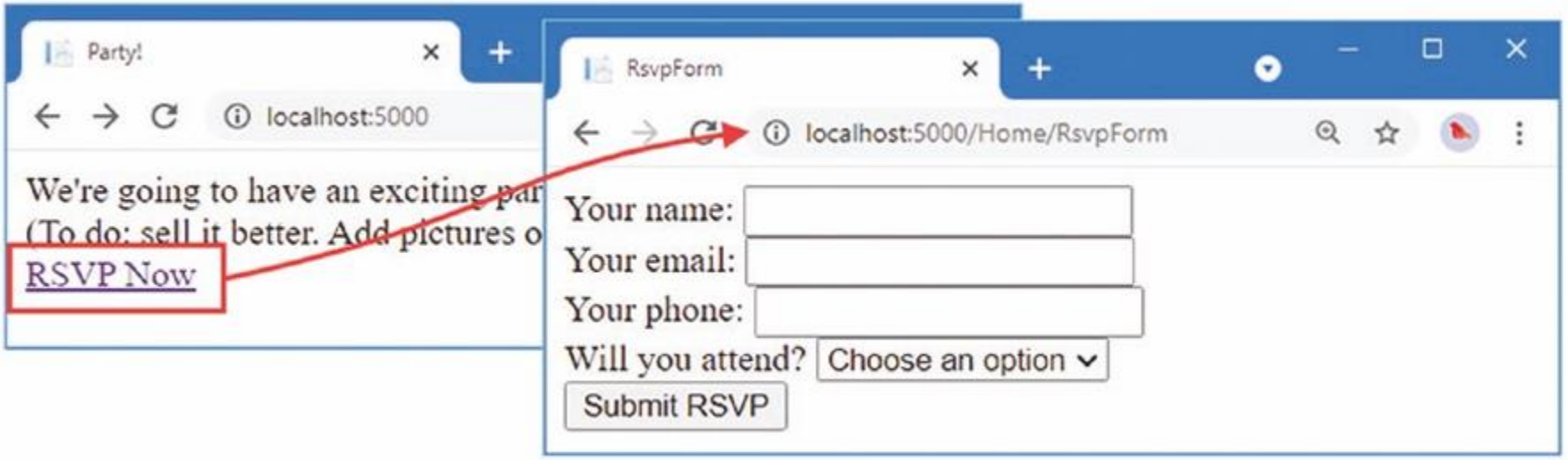
- The addition to the listing is an a element that has an **asp-action attribute**. The attribute is an example of a **tag helper attribute**, which is an instruction for Razor that will be performed when the view is rendered.
- The asp-action attribute is an instruction to add an **href** attribute to the a element that contains a URL for an action method.



BUILDING A FORM

Building the Form

- Creating a Form View in the RsvpForm.cshtml File in the Views/Home Folder



Building the Form

```
@model PartyInvites.Models.GuestResponse
@{
    Layout = null;
}
<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title>RsvpForm</title>
</head>
<body>
    <form asp-action="RsvpForm" method="post">
        <div>
            <label asp-for="Name">Your name:</label>
            <input asp-for="Name" />
        </div>
        <div>
            <label asp-for="Email">Your email:</label>
            <input asp-for="Email" />
        </div>
        <div>
            <label asp-for="Phone">Your phone:</label>
            <input asp-for="Phone" />
        </div>
        <div>
            <label asp-for="WillAttend">Will you attend?</label>
            <select asp-for="WillAttend">
                <option value="">Choose an option</option>
                <option value="true">Yes, I'll be there</option>
                <option value="false">No, I can't come</option>
            </select>
        </div>
        <button type="submit">Submit RSVP</button>
    </form>
</body>
</html>
```

```
<p>
    <label for="Name">Your name:</label>
    <input type="text" id="Name" name="Name" value="">
</p>
```

Receiving Form Data

- I have not yet told ASP.NET Core what I want to do when the form is posted to the server. As things stand, clicking the Submit RSVP button just clears any values you have entered in the form.
- That is because the form posts back to the RsvpForm action method in the Home controller, which just renders the view again

Receiving Form Data

- To receive and process submitted form data, I am going to use an important feature of controllers. I will add a second RsvpForm action method to create the following:
  - A method that responds to **HTTP GET** requests: A GET request is what a browser issues normally each time someone clicks a link. This version of the action will be responsible for displaying the **initial blank form** when someone first visits /Home/RsvpForm.
  - A method that responds to **HTTP POST** requests: The form element defined above sets the method attribute to post, which causes the form data to be sent to the server as a POST request. This version of the action will be responsible for receiving submitted data and deciding what to do with it.

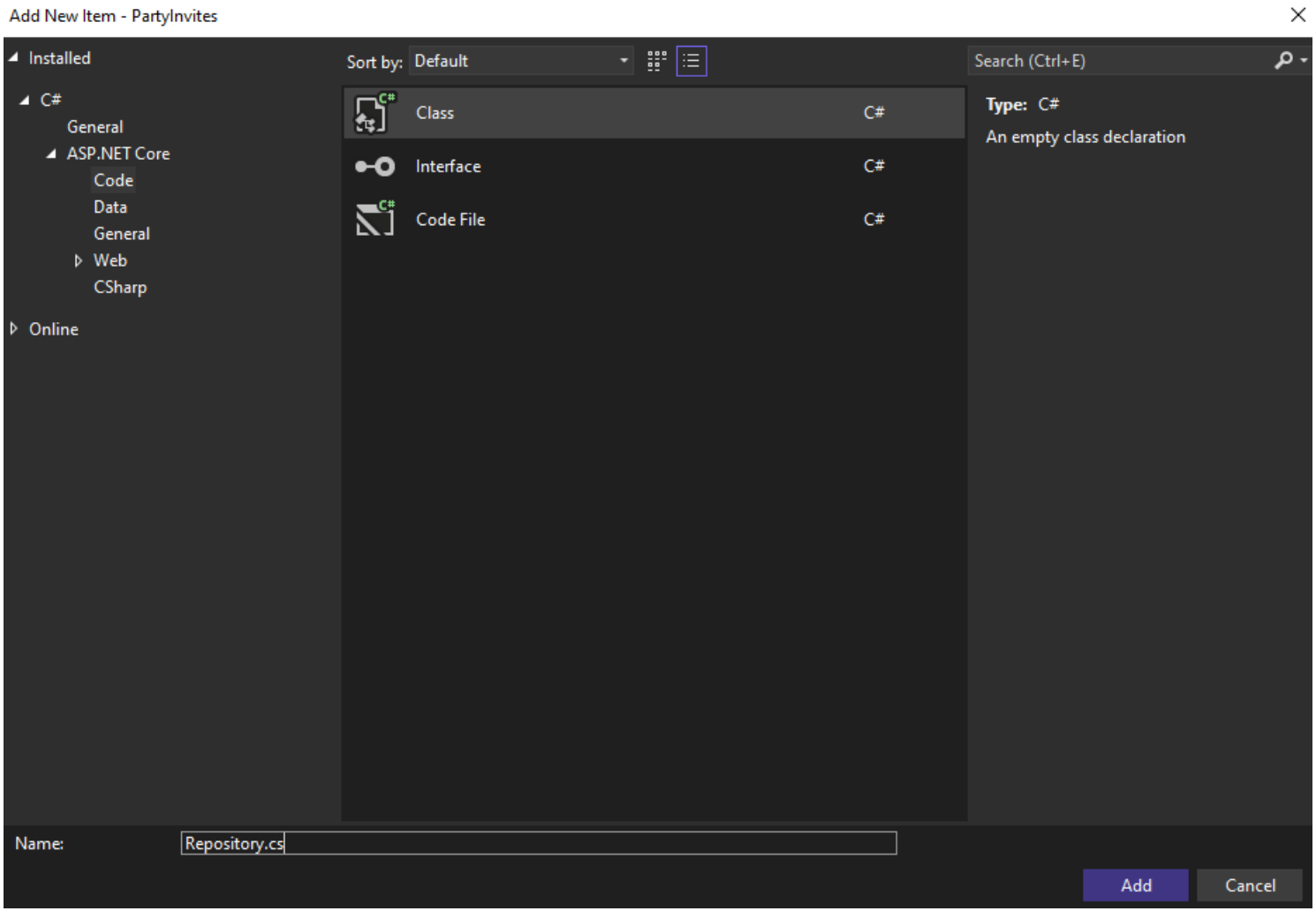
Receiving Form Data

```
using Microsoft.AspNetCore.Mvc;
using PartyInvites.Models;
namespace PartyInvites.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }

        [HttpGet]
        public ViewResult RsvpForm()
        {
            return View();
        }

        [HttpPost]
        public ViewResult RsvpForm(GuestResponse guestResponse)
        {
            // TODO: store response from guest
            return View();
        }
    }
}
```

Create Repository



Create Repository

```
namespace PartyInvites.Models
{
    public static class Repository
    {
        private static List<GuestResponse> responses = new();
        public static IEnumerable<GuestResponse> Responses => responses;
        public static void AddResponse(GuestResponse response)
        {
            Console.WriteLine(response);
            responses.Add(response);
        }
    }
}
```

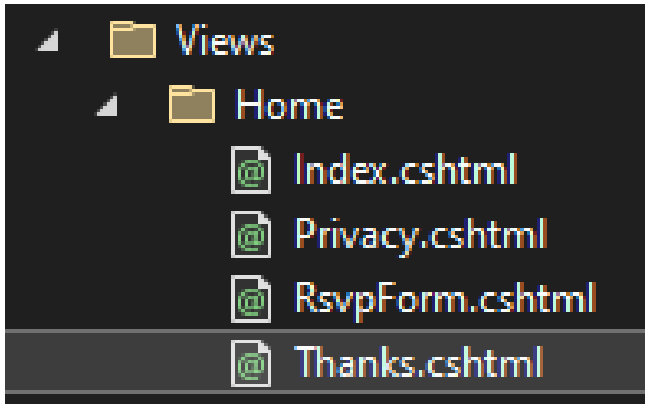
Storing Response

- Now that I have somewhere to store the data, I can update the action method that receives the HTTP POST

```
[HttpPost]
0 references
public ViewResult RsvpForm(GuestResponse guestResponse)
{
    Repository.AddResponse(guestResponse);
    return View("Thanks", guestResponse);
}
```

Adding the Thanks View

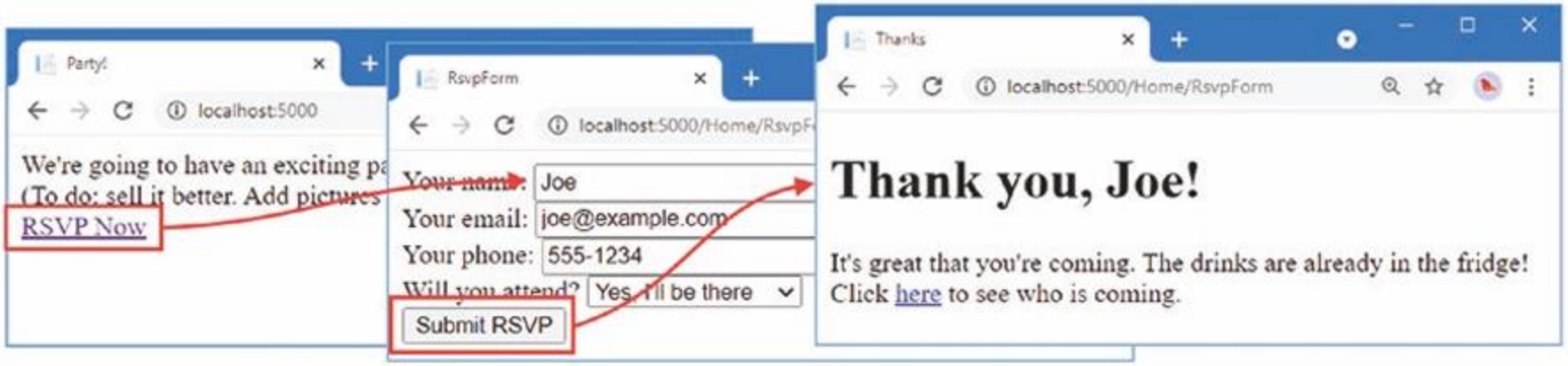
- The call to the View method in the RsvpForm action method creates a ViewResult that selects a view called Thanks and uses the GuestResponse object created by the model binder as the view model. Add a Razor View named Thanks.cshtml to the Views/Home folder



Adding the Thanks View

```
@model PartyInvites.Models.GuestResponse
@{
    Layout = null;
}
<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title>Thanks</title>
</head>
<body>
    <div>
        <h1>Thank you, @Model?.Name!</h1>
        @if (Model?.WillAttend == true)
        {
            @:It's great that you're coming. The drinks are already in the fridge!
        }
        else
        {
            @:Sorry to hear that you can't make it, but thanks for letting us know.
        }
    </div>
    Click
    <a asp-action="ListResponses">here</a> to see who is coming.
</body>
</html>
```

Adding the Thanks View



DISPLAYING THE RESPONSES

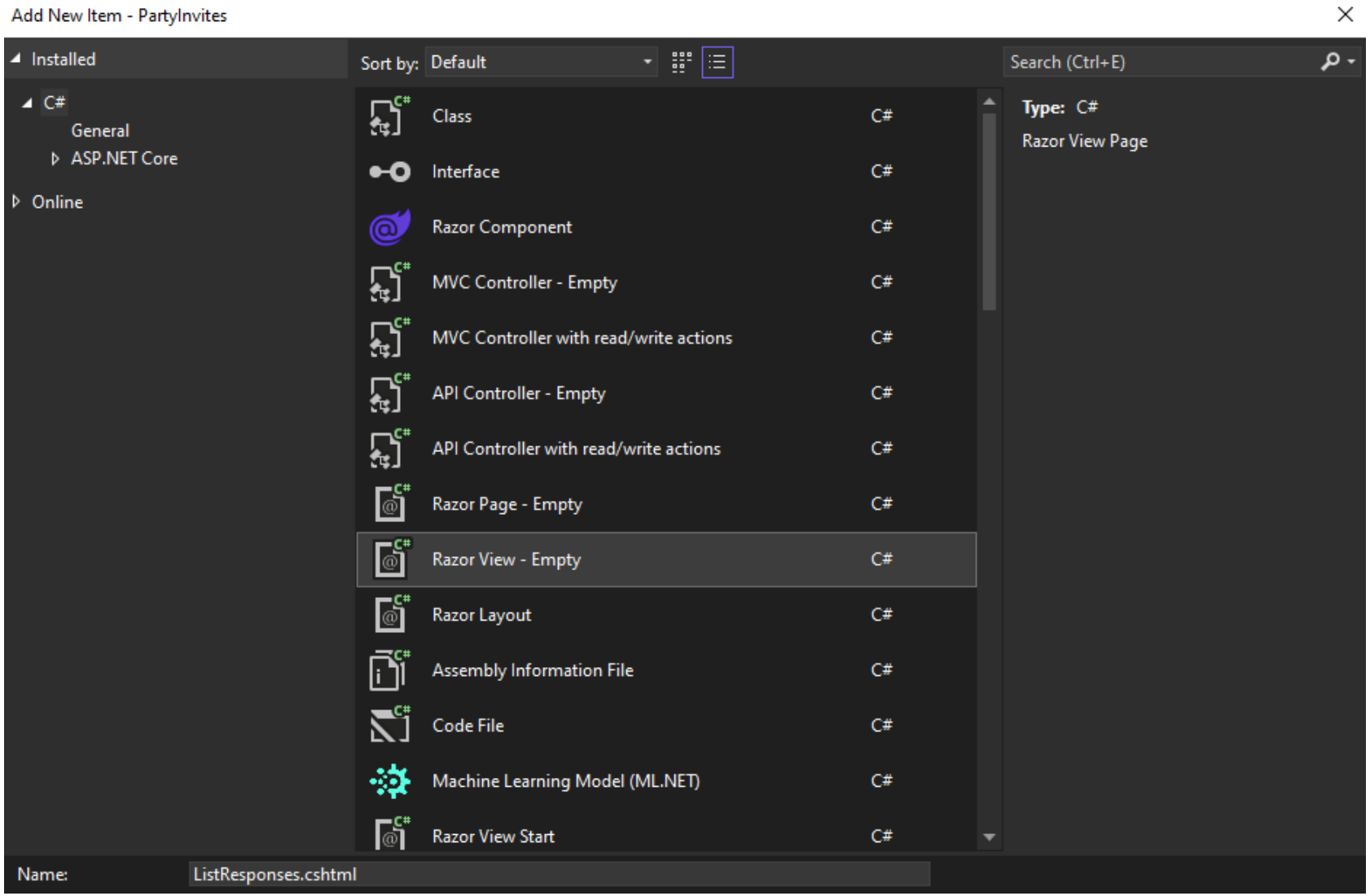
Displaying the Responses

- At the end of the Thanks.cshtml view, I added an a element to create a link to display the list of people who are coming to the party. I used the asp-action tag helper attribute to create a URL that targets an action method called ListResponses, like this:

```
...
<div>Click <a asp-action="ListResponses">here</a> to see who is coming.</div>
...
```

```
0 references
public ViewResult ListResponses()
{
    return View(Repository.Responses.Where(r => r.WillAttend == true));
}
```

Displaying the Responses



Displaying the Responses

```
@model IEnumerable<PartyInvites.Models.GuestResponse>
@{
    Layout = null;
}
<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title>Responses</title>
</head>
<body>
    <h2>Here is the list of people attending the party</h2>
    <table>
        <thead>
            <tr><th>Name</th><th>Email</th><th>Phone</th></tr>
        </thead>
        <tbody>
            @foreach (var r in Model!)
            {
                <tr>
                    <td>@r.Name</td>
                    <td>@r.Email</td>
                    <td>@r.Phone</td>
                </tr>
            }
        </tbody>
    </table>
</body>
</html>
```

VALIDATION

Validation

- I can now add data validation to the application. Without validation, users could enter nonsense data or even submit an empty form. In an ASP.NET Core application, validation rules are defined by applying attributes to model classes, which means the same validation rules can be applied in any form that uses that class.
- ASP.NET Core relies on attributes from the **System.ComponentModel.DataAnnotations** namespace

Validation

```
using System.ComponentModel.DataAnnotations;

namespace PartyInvites.Models
{
    public class GuestResponse
    {
        [Required(ErrorMessage = "Please enter your name")]
        public string? Name { get; set; }

        [Required(ErrorMessage = "Please enter your email address")]
        [EmailAddress]
        public string? Email { get; set; }
        [Required(ErrorMessage = "Please enter your phone number")]
        public string? Phone { get; set; }
        [Required(ErrorMessage = "Please specify whether you'll attend")]
        public bool? WillAttend { get; set; }
    }
}
```

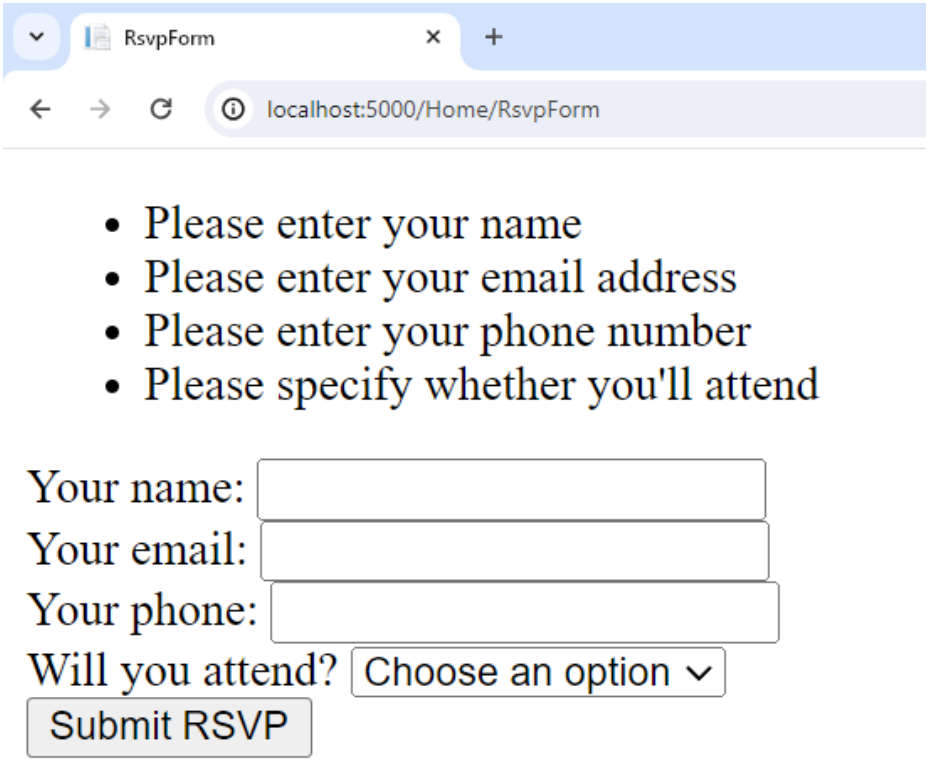
Validation

```
[HttpPost]
0 references
public ViewResult RsvpForm(GuestResponse guestResponse)
{
    if (ModelState.IsValid)
    {
        Repository.AddResponse(guestResponse);
        return View("Thanks", guestResponse);
    }
    else
    {
        return View();
    }
}
```

Check model valid or not

Validation

- When it renders a view, Razor has access to the details of any validation errors associated with the request, and tag helpers can access the details to display validation errors to the user.



Validation

```
@model PartyInvites.Models.GuestResponse
@{
    Layout = null;
}
<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title>RsvpForm</title>
</head>
<body>
    <form asp-action="RsvpForm" method="post">
        <div asp-validation-summary="All"></div>
        <div>
            <label asp-for="Name">Your name:</label>
            <input asp-for="Name" />
        </div>
```

STYLING THE CONTENT

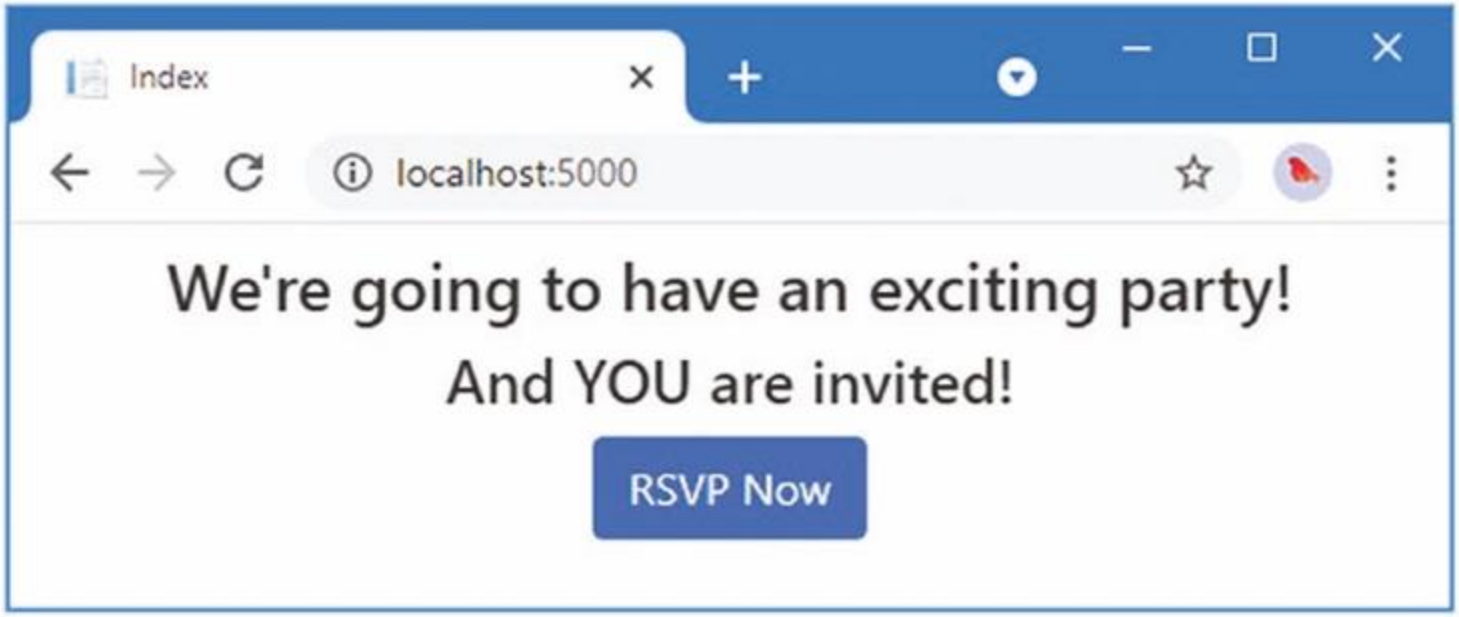
Styling the Content

- The basic Bootstrap features work by applying classes to elements that correspond to CSS selectors defined in the files added to the **wwwroot/lib/bootstrap** folder
- Here we will add Bootstrap to the Index.cshtml File in the Views/Home Folder

Styling the Content

```
@{
    Layout = null;
}
<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <link rel="stylesheet" href="/lib/bootstrap/dist/css/bootstrap.css" />
    <title>Index</title>
</head>
<body>
    <div class="text-center m-2">
        <h3> We're going to have an exciting party!</h3>
        <h4>And YOU are invited!</h4>
        <a class="btn btn-primary" asp-action="RsvpForm">RSVP Now</a>
    </div>
</body>
</html>
```

Styling the Content



Styling the Form View

- Adding Bootstrap to the RsvpForm.cshtml File in the Views/Home Folder

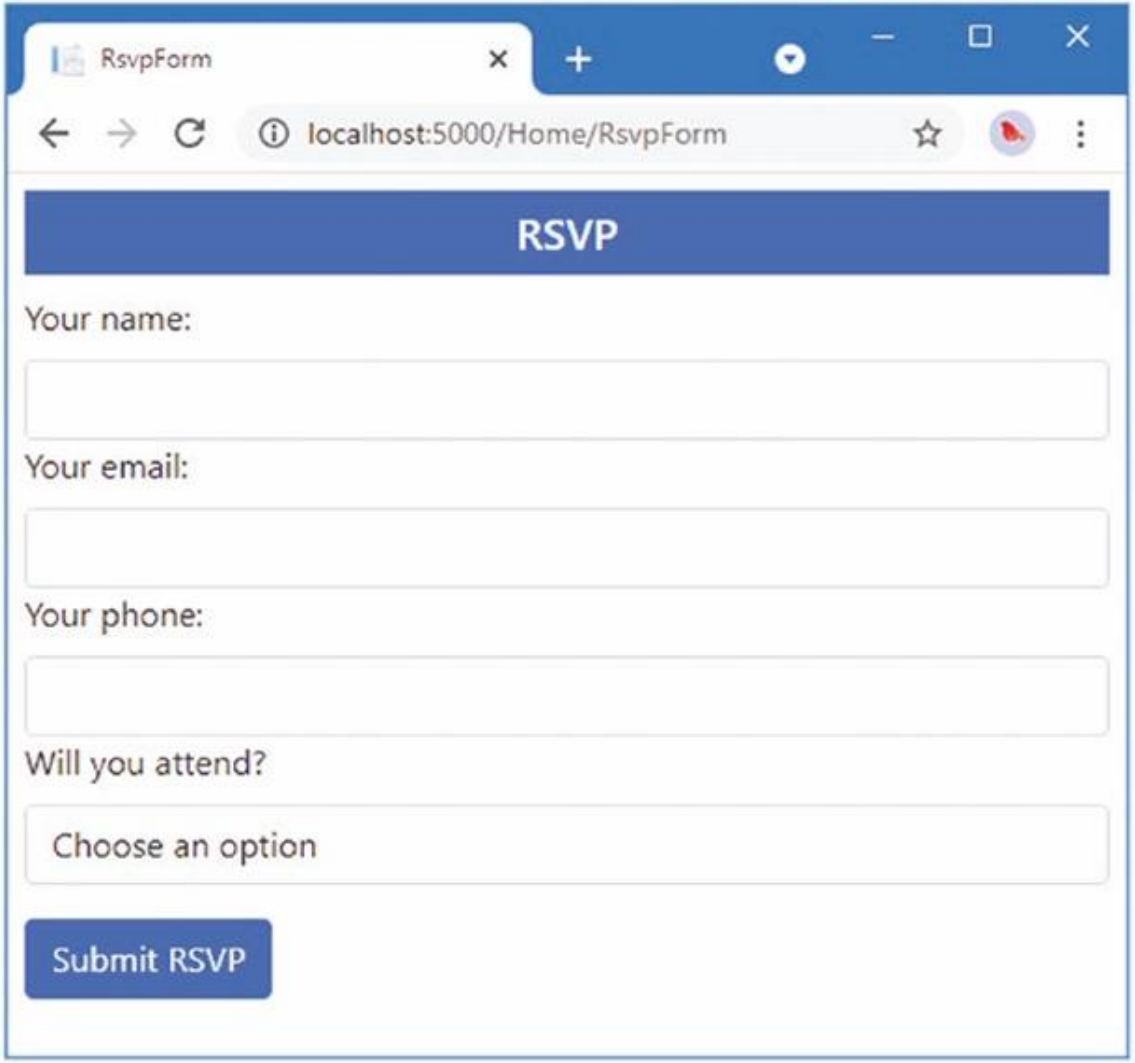
Styling the Form View

```
@model PartyInvites.Models.GuestResponse
@{
   Layout = null;
}
<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title>RsvpForm</title>
    <link rel="stylesheet" href="/lib/bootstrap/dist/css/bootstrap.css" />
    <link rel="stylesheet" href="/css/styles.css" />
</head>
<body>
    <h5 class="bg-primary text-white text-center m-2 p-2">RSVP</h5>
    <form asp-action="RsvpForm" method="post" class="m-2">
    <div asp-validation-summary="All"></div>
    <div class="form-group">
    <label asp-for="Name" class="form-label">Your name:</label>
    <input asp-for="Name" class="form-control" />
    </div>
```

Styling the Form View

```
<div class="form-group">
            <label asp-for="Email" class="form-label">Your email:</label>
            <input asp-for="Email" class="form-control" />
        </div>
        <div class="form-group">
            <label asp-for="Phone" class="form-label">Your phone:</label>
            <input asp-for="Phone" class="form-control" />
        </div>
        <div class="form-group">
            <label asp-for="WillAttend" class="form-label">
                Will you attend?
            </label>
            <select asp-for="WillAttend" class="form-control">
                <option value="">Choose an option</option>
                <option value="true">Yes, I'll be there</option>
                <option value="false">No, I can't come</option>
            </select>
        </div>
        <button type="submit" class="btn btn-primary mt-3">Submit RSVP</button>
    </form>
</body>
</html>
```

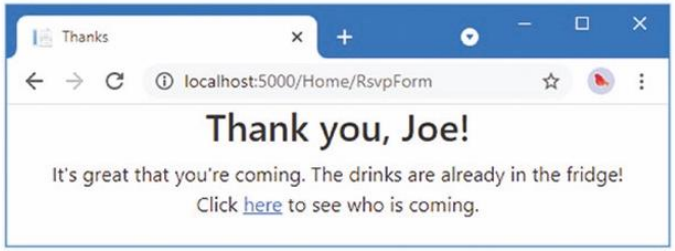
Styling the Form View



Styling the Thanks View

- The next view file to style is Thanks.cshtml

```
@model PartyInvites.Models.GuestResponse
@{
    Layout = null;
}
<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title>Thanks</title>
    <link rel="stylesheet" href="/lib/bootstrap/dist/css/bootstrap.css" />
</head>
<body class="text-center">
    <div>
        <h1>Thank you, @Model?.Name!</h1>
        @if (Model?.WillAttend == true)
        {
            @:It's great that you're coming. The drinks are already in the fridge!
        }
        else
        {
            @:Sorry to hear that you can't make it, but thanks for letting us know.
        }
    </div>
    Click
    <a asp-action="ListResponses">here</a> to see who is coming.
</body>
</html>
```



Styling the List View

- The final view to style is ListResponses, which presents the list of attendees. Styling the content follows the same approach as used for the other views

Styling the List View

```
@model IEnumerable<PartyInvites.Models.GuestResponse>
@{
    Layout = null;
}
<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title>Responses</title>
    <link rel="stylesheet" href="/lib/bootstrap/dist/css/bootstrap.css" />
</head>
<body>
    <div class="text-center p-2">
        <h2 class="text-center">
            Here is the list of people attending the party
        </h2>
        <table class="table table-bordered table-striped table-sm">
            <thead>
                <tr><th>Name</th><th>Email</th><th>Phone</th></tr>
            </thead>
            <tbody>
                @foreach (var r in Model!)
                {
                    <tr>
                        <td>@r.Name</td>
                        <td>@r.Email</td>
                        <td>@r.Phone</td>
                    </tr>
                }
            </tbody>
        </table>
    </div>
</body>
</html>
```

Styling the List View